

**UNIVERSITY INSTITUTE OF ENGINEERING AND
TECHNOLOGY PANJAB UNIVERSITY, SECTOR 25,
CHANDIGARH**

SOFTWARE TESTING AND QUALITY ASSURANCE LAB FILE

**CSE
DEPARTMENT
BATCH: 2023-27**



**SUBMITTED BY:
Tushya Gupta
UE233106
CSE Section 2
Group 6**

**SUPERVISED BY:
Mrs. Deepti Gupta**

Index

S. No.	Title	Date	Page No.	Remarks
1	Test a program to find is a triangle is equilateral, scalene or isosceles	3 Feb 2026	3 – 4	
2	Understanding different types of software testing	10 Feb 2026	5 – 9	
3	Introduction to automated testing tool - Selenium	17 Feb 2026	10 – 13	
4	Automatic browser interaction in Selenium	24 Feb 2026	14 – 15	
5	Registration form automation	24 Feb 2026	16 – 17	
6	Selenium WebDriver interactions	24 Feb 2026	18 – 34	
7	Installation of BugZilla	17 Mar 2026	35	
8	The Lifecycle of a Bug (Workflow Tracking)	24 Mar 2026	36 -38	
9	Bugzilla Basics - Task: Use a demo Bugzilla instance	7 Apr 2026	39 – 40	
10	Selenium Fundamentals - Task: Write a script to automate a login flow on a practice site.	7 Apr 2026	41	
11	Integration – The "Auto-Bug" Reporter - Task: Create a Selenium script that automatically logs into Bugzilla and files a bug report when a test fails.	7 Apr 2026	42 – 43	
12	Advanced – Data-Driven Testing & APIs - Task: Perform data-driven testing using an Excel sheet and Bugzilla's REST API.	7 Apr 2026	44 - 45	

Lab 1

Q1. Test a program to find is a triangle is equilateral, scalene or isosceles.

```
#include <iostream>
using namespace std;

int main() {
    int a = 0, b = 0, c = 0;
    cout << "Enter 1st number: ";
    cin >> a;
    if (cin.fail()) {
        cout << "Only interger type inputs allowed";
        return 1;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Enter 2nd number: ";
    cin >> b;
    if (cin.fail()) {
        cout << "Only interger type inputs allowed";
        return 1;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Enter 3rd number: ";
    cin >> c;
    if (cin.fail()) {
        cout << "Only interger type inputs allowed";
        return 1;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    // cout << a << " " << b << " " << c << endl;
    if (a <= 0 || b <= 0 || c <= 0) {
        cout << "Invlaid input";
    } else if ((long)a + b <= c || (long)b + c <= a || (long)c + a <= b) {
        cout << "Not a triangle";
    } else if (a == b && b == c) {
        cout << "Equilateral triangle";
    } else if ((a == b && a != c && b != c) || (b == c && b != a && c != a) ||
               (c == a && c != b && a != b)) {
        cout << "Isoselece triangle";
    } else if (a != b && b != c && c != a) {
        cout << "Scalene triangle";
    }
    cout << endl;
    return 0;
}
```

Test Case	Test Description	Input	Expected Output	Actual Output
TC1	Equilateral triangle	5,5,5	Equilateral triangle	Enter 1st number: 5 Enter 2nd number: 5 Enter 3rd number: 5 Equilateral triangle
TC2	Isosceles triangle	3,3,4	Isosceles triangle	Enter 1st number: 3 Enter 2nd number: 3 Enter 3rd number: 4 Isosceles triangle
TC3	Scalene triangle	2,3,4	Scalene triangle	Enter 1st number: 2 Enter 2nd number: 3 Enter 3rd number: 4 Scalene triangle
TC4	Sum of 2 sides = 3rd	2,2,4	Not a triangle	Enter 1st number: 2 Enter 2nd number: 2 Enter 3rd number: 4 Not a triangle
TC5	Sum of 2 sides < 3rd	1,2,4	Not a triangle	Enter 1st number: 1 Enter 2nd number: 2 Enter 3rd number: 4 Not a triangle
TC6	One side 0 length	0,2,4	Invalid input	Enter 1st number: 0 Enter 2nd number: 2 Enter 3rd number: 4 Invalid input
TC7	One side -ve length	-1,2,4	Invalid input	Enter 1st number: -1 Enter 2nd number: 2 Enter 3rd number: 4 Invalid input
TC8	A character input as side	a,2,3	Integer input only	Enter 1st number: a Only integer type inputs allowed
TC9	A float as input	3.5,2,4	Float converted to int	Enter 1st number: 3.5 Enter 2nd number: 2 Enter 3rd number: 4 Scalene triangle
TC10	Sum of sides greater than max size of integer	2147483647, 2147483647, 2147483647	Equilateral triangle	Enter 1st number: 2147483647 Enter 2nd number: 2147483647 Enter 3rd number: 2147483647 Equilateral triangle

Lab 2

Q2. Understanding different types of software testing.

Major categories of testing

1. Manual testing

- **Definition** - Manual testing is the process of manually testing software for defects. It requires a tester to play the role of an end user whereby they use most of the application's features to ensure correct behavior. To guarantee completeness of testing, the tester often follows a written test plan that leads them through a set of important test cases. Unlike automated testing, manual testing relies on predefined test cases and can be time-consuming and prone to human error.
- **Example** – In a hotel booking website, a tester manually searches for available rooms, selects dates, enters guest details, completes payment, and verifies that a confirmation message and email are received. The tester observes the interface, checks calculations such as total cost, and confirms that error messages appear correctly for invalid inputs.

2. Automated testing

- **Definition** - Automated testing is a software testing technique that uses specialized tools to execute test cases automatically, without human intervention. It helps validate the functionality of software quickly and efficiently, making it ideal for large or repetitive testing tasks.
- **Example** – In a web application, a tester writes a Selenium script that automatically opens the login page, enters predefined credentials, clicks the login button, and verifies that the dashboard loads successfully. This test can be run repeatedly after each code update to quickly confirm that the login functionality continues to work correctly.

Different types of testing methods

1. Black box

- **Definition** - Black-box testing is a Type of Software Testing in which the tester is not concerned with the software's internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements.
- **Example** - When testing a login page, the tester enters a valid username and password and expects successful login, then enters an incorrect password and expects an error message. The tester does not examine the authentication logic or database queries; they only validate whether the system behaves correctly based on given inputs and expected outputs.

2. White box

- **Definition** - White box testing is a software testing method that examines the internal structures or workings of an application, as opposed to its external functionality. It requires testers to have knowledge of the source code to design test cases that ensure all paths through the code are tested effectively.
- **Example** - When testing a function that calculates discounts, a developer reviews the source code and creates test cases to cover all branches—such as when the purchase amount is below the threshold, exactly at the threshold, and above it. The tester ensures that each condition, loop, and decision path in the

code executes correctly, verifying logic flow and code coverage rather than just external behavior.

3. Gray box

- **Definition** - Gray-box testing is a software testing method used to search for defects due to improper structure or application usage, where the tester has partial knowledge of the internal structure of the software. It is a combination of white-box testing, where the tester has broad access to the internal structure of the application, and black-box testing, where tester is unaware of the structure.
- **Example** - When testing a web application's password reset feature, the tester knows that reset tokens are stored in a database with an expiry time. They submit a reset request through the UI, then verify that a token is generated correctly, stored in the database, and expires after the defined duration. The tester uses partial knowledge of the system's internals to design more effective test cases while still interacting with it like an end user.

Levels of software testing

1. Functional testing

- **Definition** - Functional Testing is defined as a Type of Software Testing that verifies that each function of the Software Application works in conformance with the requirements and specifications. This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.
- **Example** - In an online banking application, a tester checks the fund transfer feature by entering valid account details and an amount within the balance limit, then confirms that the amount is deducted from the sender's account and credited to the recipient's account. The tester also verifies appropriate error messages for invalid account numbers or insufficient balance, ensuring the feature behaves as specified.

2. Non-Functional testing

- **Definition** - Non-functional testing assesses critical aspects of a software application such as usability, performance, reliability, and security. Unlike functional testing, which validates that the software functions in alignment with functional requirements, non-functional testing evaluates how well the software functions under diverse conditions. It investigates factors like speed, scalability, and user-friendliness, ensuring the software is efficient, robust, and dependable.
- **Example** - In an e-commerce website, a tester performs load testing by simulating thousands of users accessing the site simultaneously during a sale. The goal is to verify that page's load within an acceptable response time, the server does not crash, and transactions complete reliably under heavy traffic. This focuses on performance, scalability, and stability rather than specific features.

Functional Testing

1. Unit testing

- **Definition** - Unit testing is a method of software testing that focuses on evaluating the smallest parts of code, known as units, to ensure they function

correctly in isolation. This process helps identify bugs early in development, improving code quality and reducing future debugging costs.

- **Example** - In a payroll system, a developer writes a unit test for a function that calculates employee salary based on base pay, tax rate, and bonuses. The test provides fixed input values and checks whether the function returns the correct net salary. External dependencies such as databases or APIs are mocked so that only the logic of that single function is evaluated.

2. System testing

- **Definition** - System testing is testing performed on a fully integrated and complete software system to evaluate its compliance with specified requirements, typically defined in documents such as the SRS or FRS. It is conducted at the system level, after integration testing, and focuses on validating the overall behavior of the system against functional and non-functional requirements under realistic conditions.
- **Example** - In an online shopping system, testers evaluate the entire workflow—from user registration and product search to adding items to the cart, making payment, and receiving order confirmation. The goal is to confirm that all modules work together correctly as a fully integrated system under realistic conditions.

3. Integration testing

- **Definition** - Integration testing is a form of software testing in which multiple software components, modules, or services are tested together to verify they work as expected when combined. The focus is on testing the interactions and data exchange between integrated parts, rather than testing components in isolation. Integration testing describes how tests are run at the integration-level to contrast testing at the unit or system level.
- **Example** - In an online shopping application, after the payment module is integrated with the order management system, testers verify that once a payment is successfully processed, the order status is updated, inventory is reduced, and a confirmation email is triggered. The focus is on ensuring proper interaction and data flow between connected components.

4. Acceptance testing

- **Definition** - Acceptance Testing is a formal testing according to user needs, requirements, and business processes conducted to determine whether a system satisfies the acceptance criteria or not and to enable the users, customers, or other authorized entities to determine whether to accept the system or not. Acceptance Testing is the last phase of software testing performed after System Testing and before making the system available for actual use.
- **Example** - Before launching a hospital management system, hospital staff test real-world scenarios such as registering patients, scheduling appointments, generating bills, and producing reports. They confirm that the system supports their daily operations as specified in the requirements, ensuring it is fit for actual use before final approval.

5. Regression testing

- **Definition** - Regression testing is re-running functional and non-functional tests to ensure that previously developed and tested software still performs as expected after a change. If not, that would be called a regression. Changes that may require regression testing include bug fixes, software enhancements, configuration changes, and even substitution of electronic components.

- **Example** - After fixing a defect in the payment module of an online shopping application, testers re-execute previously passed test cases for login, product search, cart operations, and order processing to confirm that the recent change has not unintentionally affected other parts of the system.

6. Alpha-beta testing

- **Alpha Testing** - Alpha Testing is a type of software testing performed to identify bugs before releasing the product to real users or to the public. Alpha Testing is one of the user acceptance tests. It is the first stage of software testing, during which the internal development team tests the program before making it available to clients or people outside the company.
- **Beta Testing** - Beta Testing is performed by real users of the software application in a real environment. Beta testing is one type of User Acceptance Testing. A pre-release version of the product is made available for testing to a chosen set of external users or customers during the second phase of software testing.
- **Example** - A software company developing a mobile app first performs alpha testing internally, where employees test core features, report defects, and verify stability in a controlled environment. After major issues are resolved, the company releases a beta version to a limited group of external users who use the app in real-world conditions and provide feedback on usability, performance, and remaining defects before the final public release.

Non-Functional testing

1. Performance testing

- **Definition** - Performance Testing is a type of software testing that ensures software applications perform properly under their expected workload. It is a testing technique carried out to determine system performance in terms of sensitivity, reactivity, and stability under a particular workload.
- **Example** - In an online ticket booking system, testers simulate 5,000 users attempting to book tickets simultaneously to measure response time, transaction speed, and system stability. The objective is to ensure the application responds within acceptable time limits and maintains reliability under normal and peak usage conditions.

2. Usability testing

- **Definition** - Usability testing is a method used to assess how easily real users can navigate and interact with a website, app, or digital product. It helps identify pain points, measure task success, and improve the overall user experience.
- **Example** - A company developing a food delivery app asks a group of users to place an order without prior instructions. Observers monitor how easily users can search for restaurants, add items to the cart, and complete payment. If users struggle to find buttons or misunderstand navigation steps, the design is revised to improve clarity and user experience.

3. Security testing

- **Definition** - Security Testing is a type of Software Testing that uncovers vulnerabilities, threats, risks in a software application and prevents malicious attacks from intruders. The purpose of Security Tests is to identify all possible loopholes and weaknesses of the software system which might result in a loss of information, revenue, reputation at the hands of the employees or outsiders of the Organization.

- **Example** - In an online banking application, testers attempt to log in using SQL injection strings in the username field to check for injection flaws. They also test whether sensitive data such as passwords are encrypted during transmission and ensure that users cannot access other customers' account information without proper authorization.

4. Load testing

- **Definition** - Load testing is a process used to evaluate how a system or application performs under expected normal and peak load conditions by simulating user demand. It helps identify the system's capacity, response times, and potential breaking points to ensure stability and performance before deployment.
- **Example** - A streaming platform anticipates 10,000 concurrent users during a live event. Testers simulate this number of users accessing the platform simultaneously to measure response time, buffering rate, and server stability. The goal is to verify that the system maintains acceptable performance under normal peak load conditions.

5. Stress testing

- **Definition** - Stress testing is a form of deliberately intense or thorough testing, used to determine the stability of a given system, critical infrastructure or entity. It involves testing beyond normal operational capacity, often to a breaking point, to observe the results.
- **Example** - An online ticket booking system designed to handle 5,000 concurrent users is tested with 20,000 simultaneous users. Testers observe whether the system crashes, slows down significantly, or recovers gracefully after failure. The objective is to determine the system's breaking point and assess its stability and recovery capability under excessive load.

6. Portability testing

- **Definition** - Software testing that assesses a program's ability to function properly in many environments without requiring major changes is known as portability testing. The goal is to guarantee that the program may be transferred or ported to different platforms, operating systems, browsers, and configurations and remains functional, flexible, and compatible.
- **Example** - A desktop accounting application is installed and tested on Windows, macOS, and Linux systems to ensure that all features—such as report generation, data export, and printing—work consistently. The tester also checks that the installation process, performance, and user interface behave properly across different operating systems and hardware configurations.

Lab 3

Q3. Introduction to automated testing tool - Selenium.

Selenium is an open-source framework used for automating web browsers. It allows testers and developers to write scripts that simulate user interactions with web applications.

Installation

- Python
 - Create a virtual environment → `python3 -m venv venv`
 - Activate the virtual environment → Platform specific
 - `pip install selenium`
- WebDriver Installation
 - Automatically done by selenium when calling a particular browser type
 - Can be installed manually but you need to give its path as argument to WebDriver initialization

It supports multiple:

- **Programming languages:** Java, Python, C#, JavaScript, Ruby
- **Browsers:** Chrome, Firefox, Edge, Safari
- **Operating systems:** Windows, macOS, Linux

Selenium Components

Selenium has four Components of Selenium components, which are Selenium IDE, Selenium RC, Selenium Web-Driver, and Selenium GRID.

1. **Selenium IDE** - Selenium IDE serves as an innovative toolkit for web testing, allowing users to record interactions with web applications.
 - **Record:** With Selenium IDE, users can record how they use a web application.
 - **Playback:** Selenium IDE automatically repeats what you recorded earlier.
 - **Browser Check:** Selenium IDE works on various browsers for testing.
 - **Check Elements:** Users can easily look at different parts of a webpage and set up how to work with them.
 - **Spotting Errors:** Selenium IDE helps users find and fix issues in their automated tests, one step at a time.
 - **Exporting Tests:** You can save tests created in Selenium IDE in different programming languages (like Java, Python, or C#). This lets you use them with other Selenium tools.
2. **Selenium Web Driver** - Selenium WebDriver is a robust open-source framework for automating web browsers, primarily aimed at easing the testing and verification of web applications. As an important part of the Selenium suite, WebDriver offers a programming interface to interact with web browsers, allowing developers and testers to automate browser actions seamlessly.
 - **Direct Communication with Browsers:** Unlike Selenium RC, WebDriver interacts directly with the browser's native support for automation, leading to more stable and reliable testing.
 - **Support for Parallel Execution:** WebDriver allows for parallel test execution, enabling faster test cycles and efficient utilization of resources.

- Rich Set of APIs: WebDriver provides a comprehensive set of APIs for navigating through web pages, interacting with web elements , managing windows, handling alerts , etc.
3. **Selenium GRID** - Selenium Grid is a server that allows tests to use web browser instances running on remote machines. With Selenium Grid, one server acts as the hub. Tests contact the hub to obtain access to browser instances.
- Selenium Grid allows running tests in parallel on multiple machines and managing different browser versions.
 - The ability to run tests on remote browser instances is useful to spread the load of testing across several machines.
 - Run tests in browsers running on different platforms or operating systems.

Use Cases of Selenium

1. Functional Testing - Automates validation of application features against requirements.
 - Login functionality
 - Form validation
 - Navigation workflows
 - Button and link functionality
2. Regression Testing - Used to re-run test cases after code changes to ensure existing functionality is not broken.
3. Cross-Browser Testing - Ensures web applications behave consistently across different browsers.
4. Data-Driven Testing - Allows execution of the same test with multiple sets of input data.
5. Continuous Integration (CI/CD) - Integrated with tools like Jenkins to run automated tests whenever new code is pushed.
6. Parallel Test Execution - Using Selenium Grid to reduce test execution time by running tests simultaneously on different machines.

Advantages

- Open-source and free
- Supports multiple languages
- Works with major browsers
- Strong community support
- Integrates with TestNG, JUnit, Maven, Jenkins

Limitations

- Only supports web applications (not desktop/mobile apps directly)
- Requires programming knowledge
- No built-in reporting (needs external tools)
- Slower compared to API-level testing

Selenium commands –

- `webdriver.Chrome()` – Launches a new Chrome browser instance.
- `driver.get(url)` – Opens the specified URL in the browser.
- `driver.close()` – Closes the current browser window.
- `driver.quit()` – Closes all browser windows and ends the session.
- `driver.refresh()` – Refreshes the current page.

- `driver.back()` – Navigates one step backward in browser history.
- `driver.forward()` – Navigates one step forward in browser history.
- `driver.maximize_window()` – Maximizes the browser window.
- `driver.minimize_window()` – Minimizes the browser window.
- `driver.set_window_size(w, h)` – Sets browser window width and height.
- `driver.current_url` – Returns the current page URL.
- `driver.title` – Returns the page title.
- `driver.page_source` – Returns the page HTML source.
- `driver.find_element(By.ID, "id")` – Finds an element using ID.
- `driver.find_element(By.NAME, "name")` – Finds an element using name.
- `driver.find_element(By.XPATH, "xpath")` – Finds an element using XPath.
- `driver.find_element(By.CSS_SELECTOR, "css")` – Finds an element using CSS selector.
- `driver.find_element(By.CLASS_NAME, "class")` – Finds an element using class name.
- `driver.find_element(By.TAG_NAME, "tag")` – Finds an element using tag name.
- `driver.find_element(By.LINK_TEXT, "text")` – Finds a link using full visible text.
- `driver.find_element(By.PARTIAL_LINK_TEXT, "text")` – Finds a link using partial visible text.
- `driver.find_elements(By.*, "value")` – Returns a list of matching elements.
- `element.click()` – Clicks on the element.
- `element.send_keys("text")` – Types text into an input field.
- `element.clear()` – Clears text from an input field.
- `element.submit()` – Submits a form.
- `element.text` – Returns the visible text of the element.
- `element.get_attribute("attr")` – Returns the value of an attribute.
- `element.is_displayed()` – Checks if element is visible.
- `element.is_enabled()` – Checks if element is enabled.
- `element.is_selected()` – Checks if element is selected.
- `driver.implicitly_wait(seconds)` – Sets implicit wait time.
- `WebDriverWait(driver, t).until(condition)` – Waits until a condition is met.
- `expected_conditions.presence_of_element_located()` – Waits for element presence.
- `expected_conditions.visibility_of_element_located()` – Waits for element visibility.
- `expected_conditions.element_to_be_clickable()` – Waits until element is clickable.
- `driver.switch_to.alert` – Switches to alert box.
- `alert.accept()` – Accepts the alert.
- `alert.dismiss()` – Dismisses the alert.
- `alert.text` – Gets alert message text.
- `alert.send_keys("text")` – Sends text to alert input.
- `driver.switch_to.frame(name_or_id)` – Switches to a frame.
- `driver.switch_to.default_content()` – Switches back to main page.
- `driver.switch_to.window(handle)` – Switches to a browser window/tab.
- `driver.window_handles` – Returns list of window handles.

- `driver.current_window_handle` – Returns current window handle.
- `driver.execute_script("js")` – Executes JavaScript in browser.
- `driver.save_screenshot("file.png")` – Takes screenshot of page.

Lab 4

Q4. Automatic browser interaction in Selenium.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

# %%

driver = webdriver.Chrome()

driver.get("https://www.selenium.dev/selenium/web/web-form.html")

# %%

title = driver.title
print(title)

driver.implicitly_wait(0.5)

text_box = driver.find_element(by=By.NAME, value="my-text")
submit_button = driver.find_element(by=By.CSS_SELECTOR, value="button")

# %%

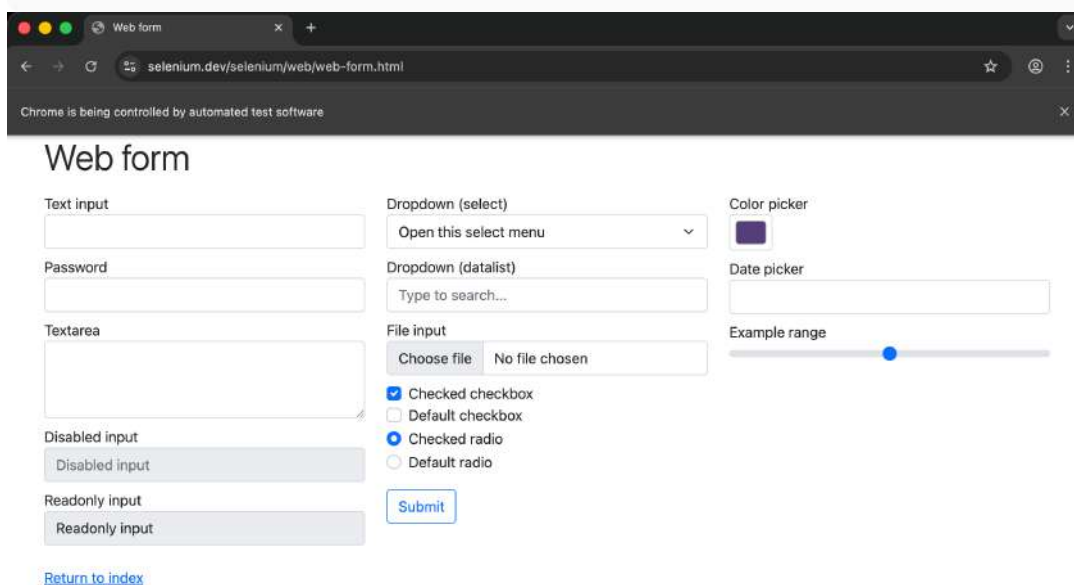
text_box.send_keys("Selenium")
submit_button.click()

# %%

message = driver.find_element(by=By.ID, value="message")
text = message.text
print(text)

# %%

driver.quit()
```



```
In [8]: print(title)
Web form
```

Web form

Text input

Password

Textarea

Disabled input

Readonly input

[Return to index](#)

```
In [17]: print(text)
Received!
```

Form submitted

Received!

Q5. Registration form automation.

```
from selenium import webdriver
from pathlib import Path
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select

###

driver = webdriver.Chrome()
driver.get("https://www.selenium.dev/selenium/web/web-form.html")

###

text_input = driver.find_element(By.ID, "my-text-id")
password_input = driver.find_element(By.NAME, "my-password")
textarea = driver.find_element(By.NAME, "my-textarea")

text_input.send_keys("Sample text")
password_input.send_keys("mypassword123")
textarea.send_keys("Testing textarea automation")

select_element = Select(driver.find_element(By.NAME, "my-select"))
select_element.select_by_visible_text("Two")

datalist = driver.find_element(By.NAME, "my-datalist")
datalist.send_keys("Seattle")

file_input = driver.find_element(By.NAME, "my-file")
file_input.send_keys(str(Path("./main.py").absolute()))

checkboxbox1 = driver.find_element(By.ID, "my-check-1")
checkboxbox2 = driver.find_element(By.ID, "my-check-2")

if not checkbox1.is_selected():
    checkbox1.click()

if not checkbox2.is_selected():
    checkbox2.click()

radio1 = driver.find_element(By.ID, "my-radio-1")
radio2 = driver.find_element(By.ID, "my-radio-2")

radio1.click()
radio2.click()

color_picker = driver.find_element(By.NAME, "my-colors")
driver.execute_script("arguments[0].value='#ff0000'", color_picker)

date_picker = driver.find_element(By.NAME, "my-date")
date_picker.send_keys("12/31/2024")
```

```
range_slider = driver.find_element(By.NAME, "my-range")
driver.execute_script("arguments[0].value=7", range_slider)

#%%

submit_button = driver.find_element(By.CSS_SELECTOR, "button")
submit_button.click()

message = driver.find_element(By.ID, "message")
print("Result:", message.text)

#%%

driver.quit()
```

Web form

Text input 	Dropdown (select) Open this select menu ▾	Color picker <input type="color"/>
Password 	Dropdown (datalist) Type to search...	Date picker
Textarea 	File input Choose file No file chosen	Example range <input type="range"/>
Disabled input Disabled input	☒ Checked checkbox ☐ Default checkbox ☒ Checked radio ☐ Default radio	
Readonly input Readonly input	Submit	

[Return to index](#)

Web form

Text input Sample text	Dropdown (select) Two ▾	Color picker <input type="color" value="#ff0000"/>
Password 	Dropdown (datalist) Seattle	Date picker 12/31/2024
Textarea Testing textarea automation	File input Choose file main.py	Example range <input type="range"/>
Disabled input Disabled input	☒ Checked checkbox ☒ Default checkbox ☐ Checked radio ☒ Default radio	
Readonly input Readonly input	Submit	

[Return to index](#)

Form submitted

Received!

Q6. Selenium WebDriver interactions.**Alerts –**

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait

global url
url = "https://www.selenium.dev/documentation/webdriver/interactions/alerts/"

def test_alert_popup():
    driver = webdriver.Chrome()
    driver.get(url)
    element = driver.find_element(By.LINK_TEXT, "See an example alert")
    element.click()

    input()
    wait = WebDriverWait(driver, timeout=2)
    alert = wait.until(lambda d : d.switch_to.alert)
    text = alert.text
    alert.accept()
    assert text == "Sample alert"

    input()
    driver.quit()

def test_confirm_popup():
    driver = webdriver.Chrome()
    driver.get(url)
    element = driver.find_element(By.LINK_TEXT, "See a sample confirm")
    driver.execute_script("arguments[0].click();", element)

    input()
    wait = WebDriverWait(driver, timeout=2)
    alert = wait.until(lambda d : d.switch_to.alert)
    text = alert.text
    alert.dismiss()
    assert text == "Are you sure?"

    input()
    driver.quit()

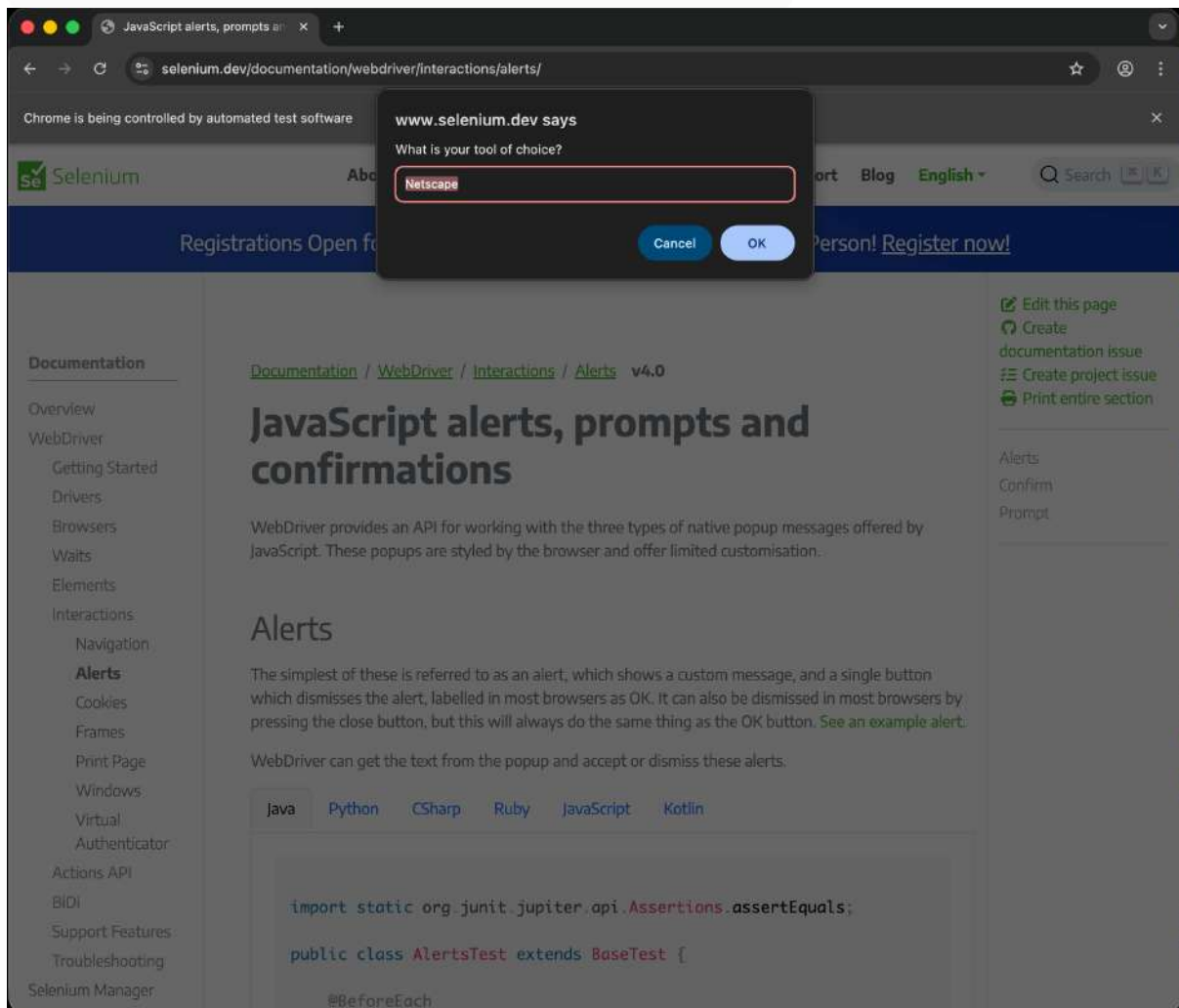
def test_prompt_popup():
    driver = webdriver.Chrome()
    driver.get(url)
    element = driver.find_element(By.LINK_TEXT, "See a sample prompt")
    driver.execute_script("arguments[0].click();", element)

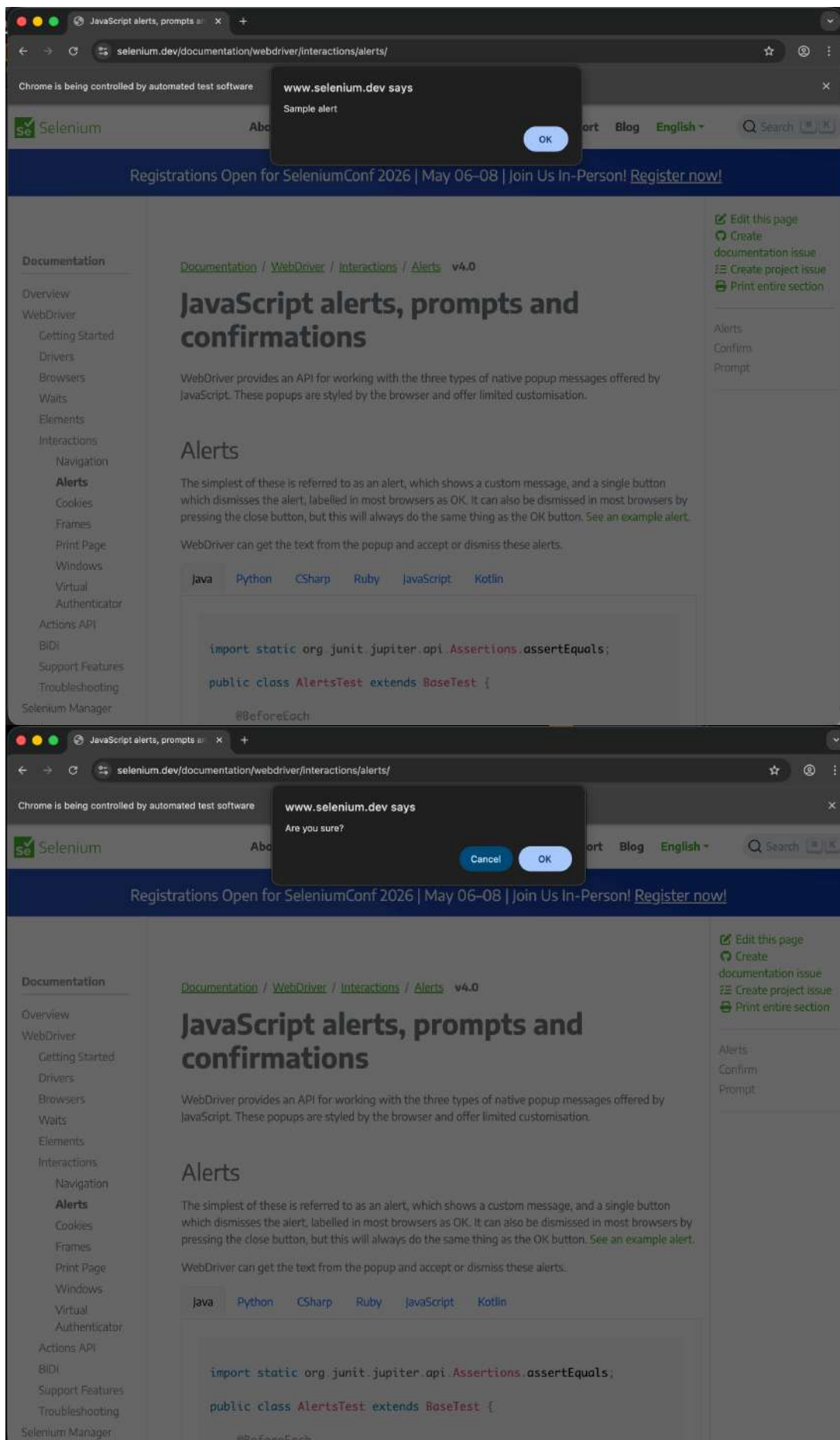
    input()
    wait = WebDriverWait(driver, timeout=2)
    alert = wait.until(lambda d : d.switch_to.alert)
    alert.send_keys("Selenium")
```

```
text = alert.text
alert.accept()
assert text == "What is your tool of choice?"

input()
driver.quit()

if __name__ == "__main__":
    test_prompt_popup()
    test_alert_popup()
    test_confirm_popup()
```





Cookies -

```
from selenium import webdriver

def test_add_cookie():
    driver = webdriver.Chrome()
    driver.get("http://www.example.com")

    # Adds the cookie into current browser context
    driver.add_cookie({"name": "key", "value": "value"})
    print(driver.get_cookies())

def test_get_named_cookie():
    driver = webdriver.Chrome()
    driver.get("http://www.example.com")

    # Adds the cookie into current browser context
    driver.add_cookie({"name": "foo", "value": "bar"})

    # Get cookie details with named cookie 'foo'
    print(driver.get_cookie("foo"))

def test_get_all_cookies():
    driver = webdriver.Chrome()

    driver.get("http://www.example.com")

    driver.add_cookie({"name": "test1", "value": "cookie1"})
    driver.add_cookie({"name": "test2", "value": "cookie2"})

    # Get all available cookies
    print(driver.get_cookies())

def test_delete_cookie():
    driver = webdriver.Chrome()

    driver.get("http://www.example.com")

    driver.add_cookie({"name": "test1", "value": "cookie1"})
    driver.add_cookie({"name": "test2", "value": "cookie2"})

    # Delete cookie with name 'test1'
    driver.delete_cookie("test1")
    print(driver.get_cookies())

def test_delete_all_cookies():
    driver = webdriver.Chrome()
```

```

driver.get("http://www.example.com")

driver.add_cookie({"name": "test1", "value": "cookie1"})
driver.add_cookie({"name": "test2", "value": "cookie2"})

# Delete all cookies
driver.delete_all_cookies()
print(driver.get_cookies())

def test_same_side_cookie_attr():
    driver = webdriver.Chrome()

    driver.get("http://www.example.com")

    # Adds the cookie into current browser context with sameSite 'Strict' (or) 'Lax'
    driver.add_cookie({"name": "foo", "value": "value", "sameSite": "Strict"})
    driver.add_cookie({"name": "foo1", "value": "value", "sameSite": "Lax"})

    cookie1 = driver.get_cookie("foo")
    cookie2 = driver.get_cookie("foo1")

    print(cookie1)
    print(cookie2)

if __name__ == "__main__":
    test_delete_all_cookies()
    test_same_side_cookie_attr()
    test_delete_cookie()
    test_get_all_cookies()
    test_get_named_cookie()

```

Delete all cookies

[]

Same side cookies

```

{
    'domain': 'www.example.com',
    'httpOnly': False,
    'name': 'foo',
    'path': '/',
    'sameSite': 'Strict',
    'secure': True,
    'value': 'value'
}
{
    'domain': 'www.example.com',
    'httpOnly': False,
    'name': 'foo1',
    'path': '/',
    'sameSite': 'Lax',
    'secure': True,
    'value': 'value'
}

```

Delete 1 cookie

```

[
    {
        'domain': 'www.example.com',
        'httpOnly': False,
        'name': 'test2',
        'path': '/',
        'sameSite': 'Lax',
        'secure': True,
        'value': 'cookie2'
    }
]

```

Get all cookies

```

[
    {
        'domain': 'www.example.com',
        'httpOnly': False,
        'name': 'test1',
        'path': '/',
        'sameSite': 'Lax',
        'secure': True,
        'value': 'cookie1'
    },
    {
        'domain': 'www.example.com',
        'httpOnly': False,
        'name': 'test2',
        'path': '/',
        'sameSite': 'Lax',
        'secure': True,
        'value': 'cookie2'
    }
]

```

Get named cookie

```

{
    'domain': 'www.example.com',
    'httpOnly': False,
    'name': 'foo',
    'path': '/',
    'sameSite': 'Lax',
    'secure': True,
    'value': 'bar'
}

```

Frames –

```
# Licensed to the Software Freedom Conservancy (SFC) under one
# or more contributor license agreements. See the NOTICE file
# distributed with this work for additional information
# regarding copyright ownership. The SFC licenses this file
# to you under the Apache License, Version 2.0 (the
# "License"); you may not use this file except in compliance
# with the License. You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
# Unless required by applicable law or agreed to in writing,
# software distributed under the License is distributed on an
# "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
# KIND, either express or implied. See the License for the
# specific language governing permissions and limitations
# under the License.

from selenium import webdriver
from selenium.webdriver.common.by import By

# set chrome and launch web page
driver = webdriver.Chrome()
driver.get("https://www.selenium.dev/selenium/web/iframes.html")

# --- Switch to iframe using WebElement ---
iframe = driver.find_element(By.ID, "iframe1")
driver.switch_to.frame(iframe)
assert "We Leave From Here" in driver.page_source

email_element = driver.find_element(By.ID, "email")
email_element.send_keys("admin@selenium.dev")
email_element.clear()
driver.switch_to.default_content()

# --- Switch to iframe using name or ID ---
iframe1 = driver.find_element(
    By.NAME, "iframe1-name"
) # (This line doesn't switch, just locates)
driver.switch_to.frame(iframe1)
assert "We Leave From Here" in driver.page_source

email = driver.find_element(By.ID, "email")
email.send_keys("admin@selenium.dev")

# %%

email.clear()
driver.switch_to.default_content()

# --- Switch to iframe using index ---
driver.switch_to.frame(0)
```

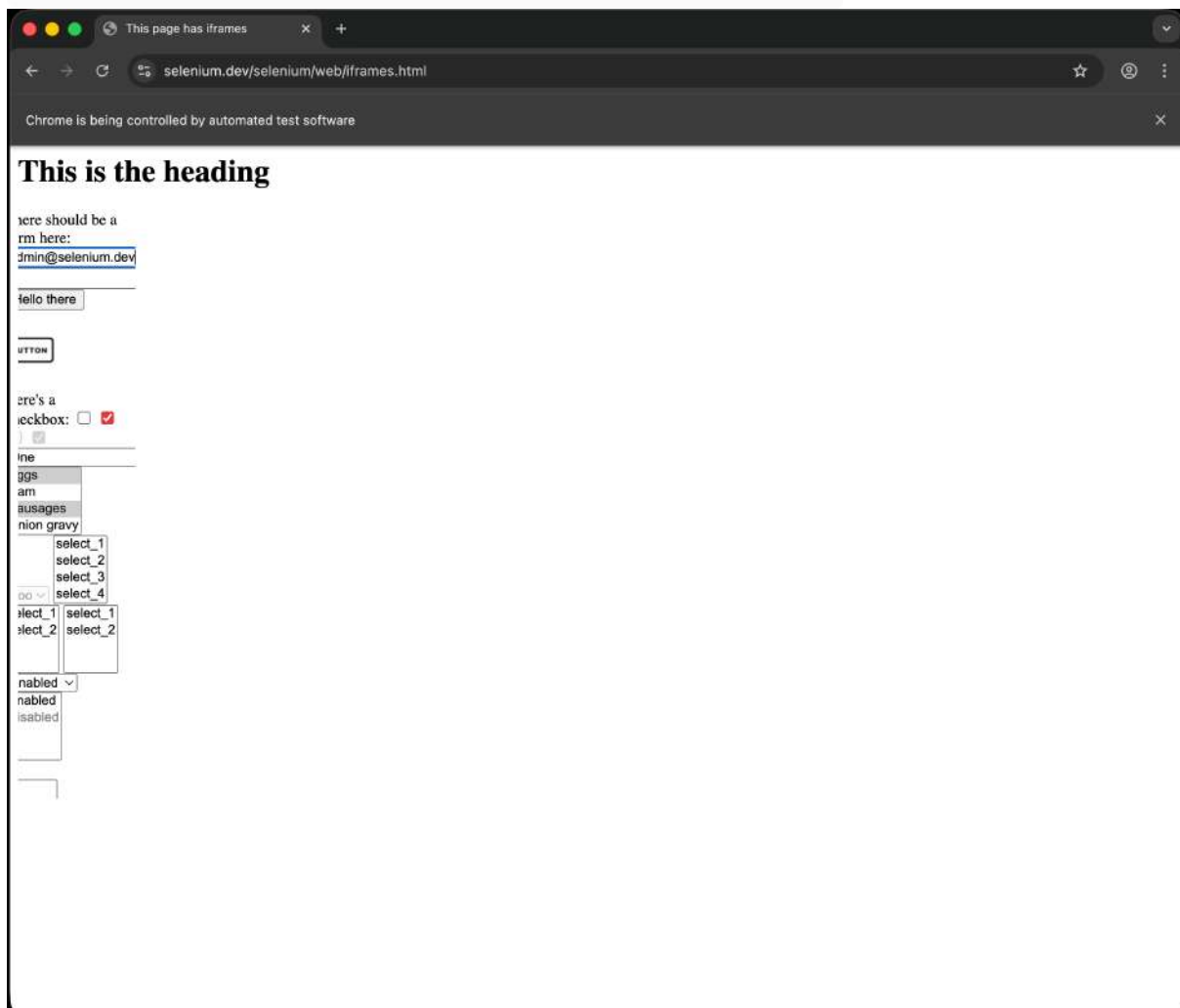
```
assert "We Leave From Here" in driver.page_source

# --- Final page content check ---
driver.switch_to.default_content()
assert "This page has iframes" in driver.page_source

# %%

# quit the driver
driver.quit()

# demo code for conference
```



Test Interactions –

```
from selenium import webdriver

#%%

driver = webdriver.Chrome()

driver.get("https://www.selenium.dev")

#%%

title = driver.title
print(title)
assert title == "Selenium"

url = driver.current_url
print(url)
assert url == "https://www.selenium.dev/"

#%%

driver.quit()
```

```
In [6]: title = driver.title
In [7]: print(title)
Selenium
In [8]: assert title == "Selenium"
In [9]: url = driver.current_url
In [10]: print(url)
https://www.selenium.dev/
In [11]: assert url == "https://www.selenium.dev/"
```

Navigation –

```
from selenium import webdriver

###

driver = webdriver.Chrome()

driver.get("https://www.selenium.dev")
driver.get("https://www.selenium.dev/selenium/web/index.html")

###

title = driver.title
assert title == "Index of Available Pages"

###

driver.back()
title = driver.title
assert title == "Selenium"

###

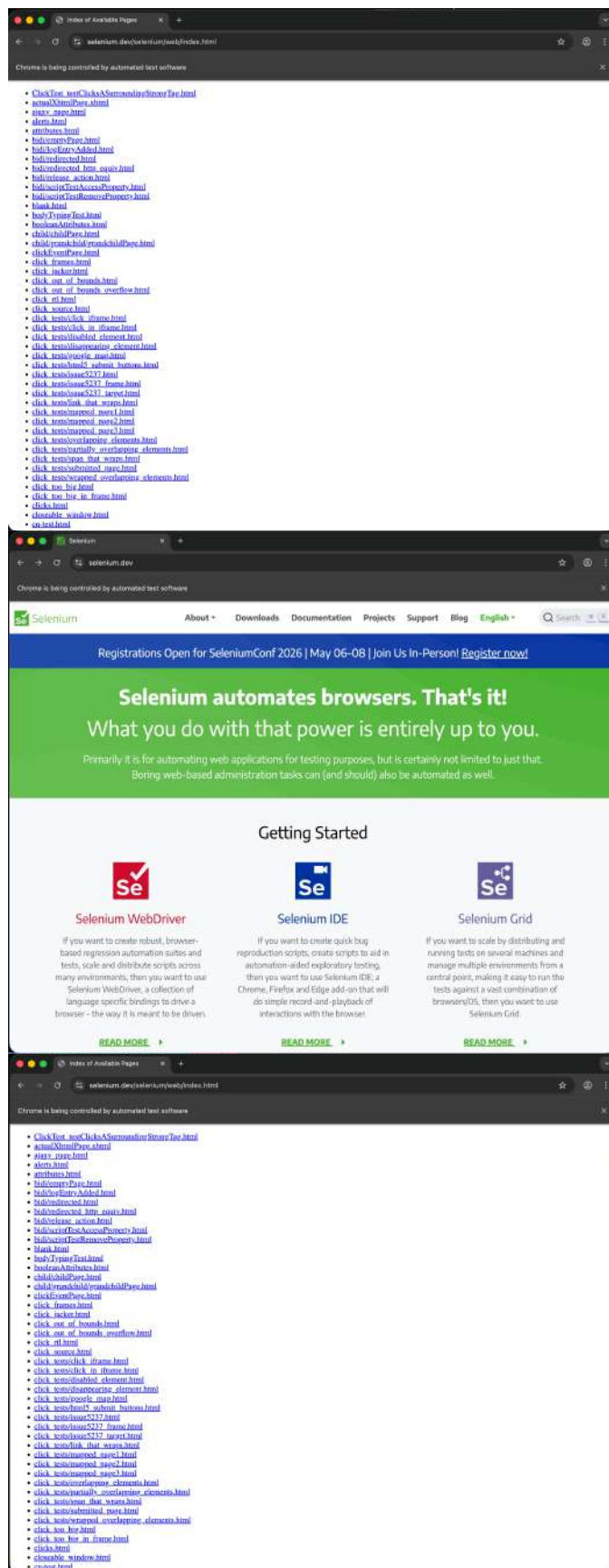
driver.forward()
title = driver.title
assert title == "Index of Available Pages"

###

driver.refresh()
title = driver.title
assert title == "Index of Available Pages"

###

driver.quit()
```



Print Options –

```
from selenium import webdriver
from selenium.webdriver.common.print_page_options import PrintOptions

def test_orientation():
    driver = webdriver.Chrome()
    driver.get("https://www.selenium.dev/")
    print_options = PrintOptions()
    print_options.orientation = "landscape" ## landscape or portrait
    assert print_options.orientation == "landscape"
    driver.quit()

def test_range():
    driver = webdriver.Chrome()
    driver.get("https://www.selenium.dev/")
    print_options = PrintOptions()
    print_options.page_ranges = ["1, 2, 3"] ## ["1", "2", "3"] or ["1-3"]
    assert print_options.page_ranges == ["1, 2, 3"]
    driver.quit()

def test_size():
    driver = webdriver.Chrome()
    driver.get("https://www.selenium.dev/")
    print_options = PrintOptions()
    print_options.page_height = 27.94 # Use page_width to assign width
    assert print_options.page_height == 27.94
    driver.quit()

def test_margin():
    driver = webdriver.Chrome()
    driver.get("https://www.selenium.dev/")
    print_options = PrintOptions()
    print_options.margin_top = 10
    print_options.margin_bottom = 10
    print_options.margin_left = 10
    print_options.margin_right = 10
    assert print_options.margin_top == 10
    assert print_options.margin_bottom == 10
    assert print_options.margin_left == 10
    assert print_options.margin_right == 10
    driver.quit()

def test_scale():
    driver = webdriver.Chrome()
    driver.get("https://www.selenium.dev/")
    print_options = PrintOptions()
    print_options.scale = 0.5 ## 0.1 to 2.0
    current_scale = print_options.scale
    assert current_scale == 0.5
    driver.quit()
```

```
def test_background():
    driver = webdriver.Chrome()
    driver.get("https://www.selenium.dev/")
    print_options = PrintOptions()
    print_options.background = True ## True or False
    assert print_options.background is True
    driver.quit()

def test_shrink_to_fit():
    driver = webdriver.Chrome()
    driver.get("https://www.selenium.dev/")
    print_options = PrintOptions()
    print_options.shrink_to_fit = True ## True or False
    assert print_options.shrink_to_fit is True
    driver.quit()

if __name__ == "__main__":
    test_shrink_to_fit()
    test_background()
    test_scale()
    test_margin()
    test_range()
    test_orientation()
    test_size()
```

Print Page –

```
from selenium import webdriver
from selenium.webdriver.common.print_page_options import PrintOptions

def test_prints_page():
    driver = webdriver.Chrome()
    driver.get("https://www.selenium.dev/")
    print_options = PrintOptions()
    pdf = driver.print_page(print_options)
    input()
    assert len(pdf) > 0

test_prints_page()
```

Virtual Authenticator –

```

from base64 import urlsafe_b64decode, urlsafe_b64encode

import pytest
from selenium.common.exceptions import InvalidArgumentException
from selenium.webdriver.chrome.webdriver import WebDriver
from selenium.webdriver.common.virtual_authenticator import (
    Credential, VirtualAuthenticatorOptions)

BASE64__ENCODED_PK = """
MIIeVAIBADANBgkqhkiG9w0BAQEFAASCbKYYwggSiAgEAAoIBAQBDB0u5Lhs4vpowbCnmCyLUpIE7JM9sm9QXzye2G+jr+Kr
MsinWohEce478FPJLTaDzHSvOW2eeunB089ZcVvVc8RLz4qyQ8r098xS1jtgqi1NcBPETDrtzth0Du/gd0sjB2Tk3TLuBGV
oPXt54a+0o4JbBj6h3s0+5eAfGplCbSNq6hN3Jh9Y0Tw5ZA6GCEy5l8zBa0gjXytd2v20dSVoEDNiNQrkjJd2rmS2oi9AyQ
FR3B7BrPSiDlCcITZFOWglF5C31Wp/PSHwQhlnh7/6YhnE2y9tzsUvzx0wJXrBADW13+oMxrneDK3WGbXTNYgIi1PvSqXlq
GjHtCK+R2QkXAgMBAAECggEAVc6bu7VAnP6v0gD0eX4razv4FX/adCao9ZsHZ+WPX8PQxtmWYqkH5CY4TSfsuizAgyPuQ0
+j4Vjssr9VODLqFoanspt6YXsVaKannCuybasNgUJnfnLnw3an2XpU2XdmXTNYckCPRX9nsAAURWT3/n9ljc/XY22ecYxM
8sDwnHu2uKZ1B7M3X60bQYL5T/LVXkKdD6xgSNLeP4AkRx0H4egaop68hoW8FIwmDPVWYVAv08etzWCtibRXz5FcNld9MgD
/Ai7ycKy4Q1Kh5G8FI79MVVaHkSQfXPHpr7/XcmpQOEAr+BMPon4s4vnKqAGdGB3j/E3d/+4F2swykoQKBgQD8hCsp6FIQ
5umJlk9/j/nGsML85LgLaNVYpWLPKPC54YNumtvj5vx1BG+zMbT7qIE3nmUPTCHP7qb5ERZG4CdMCS6S64/qzZEqiJLCqe
pwj6j4fV5SyPWEcpxf6ehNdmcfgzVB3WolfwH1ydhx/96L1jHJcTKchdJJzlfTvq8wwKBgQDeCnKws1t5GapfE1rmC/h4oL
L2qZTth9oQmbrXYohVnoqNFsLda43ePZwL9Jmd9kYb0ax0TNMmyrP0NTj41uCFgDS0cJnNTc63ojKjegxHIYyDKRZNVUR/d
xAYB/vPfBYZUS7M89p06LLsHhZS3qpu3/hppo/Uc/AM/r8PSfLNHQBKBDnWgBh60QncChPUL0Lv9FMZPR1ZOfqLCYrjYEqi
uzGm6iKM13zXF04AGAxu1P/IA5BovFcTpg79Z8tWqZaUuWvscnl+cRlj+mMXAmdqCe08VAS0mqM1ml667axeZDIR867ZG8
K5V029Wg+4qtX5uFypNAAi6GfHkxIKrD04yOHAoGACdh4wXESi0oiDdkz3K0HPwIjn6BhZC7z8mx+pnJODU3cYukxv3WTct
lUhAsyjiQ/0bK1yX87ulqFVg00Knmh+wNaJrb9wi0NAJTMICG7tiWJ0m7fw5cftJwWkBWYADmkfTRmHDvqzQSSvoC2S7aa
9QulbC3C/qgGFNrcWgcT9KcGyAZTa1P9bFCDU7hJc2mHwJwAW7/FQKEJg8SL33KINpLwcR8fqaY0dAHWWz636osVEqosRrH
zJOGpf9x2RSWzQJ+dq8+6fACgFZOVpN644+sAHfNPAI/gnNKU50fUv+eav8f8nzlf1A3y3GIkyMyzFN3DE7e0n/lyqxE4H
BYGpI8g==
"""

@pytest.fixture(scope="module", autouse=True)
def driver():
    yield WebDriver()

def test_virtual_authenticator_options():
    options = VirtualAuthenticatorOptions()
    options.is_user_verified = True
    options.has_user_verification = True
    options.is_user_consenting = True
    options.transport = VirtualAuthenticatorOptions.Transport.USB
    options.protocol = VirtualAuthenticatorOptions.Protocol.U2F
    options.has_resident_key = False

    assert len(options.to_dict()) == 6

def test_add_authenticator(driver):
    # Create virtual authenticator options
    options = VirtualAuthenticatorOptions()

```

```
options.protocol = VirtualAuthenticatorOptions.Protocol.U2F
options.has_resident_key = False

# Register a virtual authenticator
driver.add_virtual_authenticator(options)

# Get list of credentials
credential_list = driver.get_credentials()

assert len(credential_list) == 0

def test_remove_authenticator(driver):
    # Create default virtual authenticator option
    options = VirtualAuthenticatorOptions()

    # Register a virtual authenticator
    driver.add_virtual_authenticator(options)

    # Remove virtual authenticator
    driver.remove_virtual_authenticator()
    assert driver.virtual_authenticator_id is None

def test_create_and_add_resident_key(driver):
    # Create virtual authenticator options
    options = VirtualAuthenticatorOptions()
    options.protocol = VirtualAuthenticatorOptions.Protocol.CTAP2
    options.has_resident_key = True
    options.has_user_verification = True
    options.is_user_verified = True

    # Register a virtual authenticator
    driver.add_virtual_authenticator(options)

    # parameters for Resident Credential
    credential_id = bytearray([1, 2, 3, 4])
    rp_id = "localhost"
    user_handle = bytearray([1])
    privatekey = urlsafe_b64decode(BASE64__ENCODED_PK)
    sign_count = 0

    # create a resident credential using above parameters
    resident_credential = Credential.create_resident_credential(
        credential_id, rp_id, user_handle, privatekey, sign_count
    )

    # add the credential created to virtual authenticator
    driver.add_credential(resident_credential)

    # get list of all the registered credentials
    credential_list = driver.get_credentials()
    assert len(credential_list) == 1

def test_add_resident_credential_not_supported_when_authenticator_uses_u2f_protocol(
    driver,
):
    # Create virtual authenticator options
    options = VirtualAuthenticatorOptions()
```

```
options.protocol = VirtualAuthenticatorOptions.Protocol.U2F
options.has_resident_key = False

# Register a virtual authenticator
driver.add_virtual_authenticator(options)

# parameters for Resident Credential
credential_id = bytearray({1, 2, 3, 4})
rp_id = "localhost"
user_handle = bytearray({1})
privatekey = urlsafe_b64decode(BASE64__ENCODED_PK)
sign_count = 0

# create a resident credential using above parameters
credential = Credential.create_resident_credential(
    credential_id, rp_id, user_handle, privatekey, sign_count
)

# Expect InvalidArgumentException
with pytest.raises(InvalidArgumentException):
    driver.add_credential(credential)

def test_create_and_add_non_resident_key(driver):
    # Create virtual authenticator options
    options = VirtualAuthenticatorOptions()
    options.protocol = VirtualAuthenticatorOptions.Protocol.U2F
    options.has_resident_key = False

    # Register a virtual authenticator
    driver.add_virtual_authenticator(options)

    # parameters for Non Resident Credential
    credential_id = bytearray({1, 2, 3, 4})
    rp_id = "localhost"
    privatekey = urlsafe_b64decode(BASE64__ENCODED_PK)
    sign_count = 0

    # create a non resident credential using above parameters
    credential = Credential.create_non_resident_credential(
        credential_id, rp_id, privatekey, sign_count
    )

    # add the credential created to virtual authenticator
    driver.add_credential(credential)

    # get list of all the registered credentials
    credential_list = driver.get_credentials()
    assert len(credential_list) == 1

def test_get_credential(driver):
    # Create virtual authenticator options
    options = VirtualAuthenticatorOptions()
    options.protocol = VirtualAuthenticatorOptions.Protocol.CTAP2
    options.has_resident_key = True
    options.has_user_verified = True
    options.is_user_verified = True

    # Register a virtual authenticator
```

```
driver.add_virtual_authenticator(options)

# parameters for Resident Credential
credential_id = bytearray({1, 2, 3, 4})
rp_id = "localhost"
user_handle = bytearray({1})
privatekey = urlsafe_b64decode(BASE64__ENCODED_PK)
sign_count = 0

# create a resident credential using above parameters
credential = Credential.create_resident_credential(
    credential_id, rp_id, user_handle, privatekey, sign_count
)

# add the credential created to virtual authenticator
driver.add_credential(credential)

# get list of all the registered credentials
credential_list = driver.get_credentials()

assert len(credential_list) == 1
assert credential_list[0].id == urlsafe_b64encode(credential_id).decode()

def test_remove_credential(driver):
    # Create default virtual authenticator options
    options = VirtualAuthenticatorOptions()

    # Register a virtual authenticator
    driver.add_virtual_authenticator(options)

    # parameters for Non Resident Credential
    credential_id = bytearray({1, 2, 3, 4})
    rp_id = "localhost"
    privatekey = urlsafe_b64decode(BASE64__ENCODED_PK)
    sign_count = 0

    # create a non resident credential using above parameters
    credential = Credential.create_non_resident_credential(
        credential_id, rp_id, privatekey, sign_count
    )

    # add the credential created to virtual authenticator
    driver.add_credential(credential)

    # remove the credential created from virtual authenticator
    driver.remove_credential(credential.id)

    # credential can also be removed using Byte Array
    # driver.remove_credential(credential_id)

    assert len(driver.get_credentials()) == 0

def test_remove_all_credentials(driver):
    # Create default virtual authenticator options
    options = VirtualAuthenticatorOptions()
    options.has_resident_key = True

    # Register a virtual authenticator
```

```
driver.add_virtual_authenticator(options)

# parameters for Resident Credential
credential_id = bytearray({1, 2, 3, 4})
rp_id = "localhost"
user_handle = bytearray({1})
privatekey = urlsafe_b64decode(BASE64__ENCODED_PK)
sign_count = 0

# create a resident credential using above parameters
resident_credential = Credential.create_resident_credential(
    credential_id, rp_id, user_handle, privatekey, sign_count
)

# add the credential created to virtual authenticator
driver.add_credential(resident_credential)

# remove all credentials in virtual authenticator
driver.remove_all_credentials()

assert len(driver.get_credentials()) == 0

def test_set_user_verified():
    # Create virtual authenticator options
    options = VirtualAuthenticatorOptions()
    options.is_user_verified = True

    assert options.to_dict().get("isUserVerified") is True
```

```
===== test session starts =====
platform darwin -- Python 3.12.12, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/tushya/programs/college/software_testing/selenium_web_testing
configfile: pyproject.toml
collected 10 items

lab4/test_virtual_authenticator.py ..... [100%]

===== 10 passed in 2.40s =====
```

Lab 5

Q7. Installation of BugZilla.

Bugzilla is an open-source bug tracking system widely used for managing software defects and development tasks. Installing Bugzilla typically involves preparing a web server environment with components such as Apache HTTP Server, MariaDB (or MySQL), and Perl with the required modules.

The installation process begins by obtaining the Bugzilla source code and configuring the database where bug reports and user data will be stored. The system administrator then runs the checksetup.pl script, which verifies dependencies, installs missing Perl modules, and initializes the database schema. Configuration parameters such as the site URL, administrator account, and database credentials are defined during this stage.

After configuration, the web server is started and Bugzilla becomes accessible through a browser interface. Administrators can log in to complete the setup, manage users, and configure project components. Once installed, Bugzilla provides features such as issue reporting, workflow management, attachments, and integration with development tools, enabling teams to track and resolve software defects efficiently.

Steps –

1. git clone --branch release-X.X-stable <https://github.com/bugzilla/bugzilla>
 - a. Replace X.X with version(eg 5.2)
2. Install additional dependencies(mysql, apache ...)
3. Cd into Bugzilla folder
4. perl checksetup.pl --check-modules
5. perl install-module.pl --all
6. cd /Library/WebServer/Documents
7. sudo ln -s \$HOME/bugzilla Bugzilla
8. perl checksetup.pl
9. Change DB entries in the localconfig file generated
10. perl checksetup.pl
11. perl testserver.pl http://<your-bugzilla-server>/

Lab 6

Q8. The Lifecycle of a Bug (Workflow Tracking)

This experiment helps you understand the different statuses a bug goes through from discovery to resolution.

The Task: Create a bug and move it through every possible stage in the default Bugzilla workflow.

Steps:

Report : Create a new bug with a description, severity (e.g., "Major"), and priority (e.g., "P1").

Assign : Change the status from UNCONFIRMED or NEW to ASSIGNED.

Resolve : Mark the bug as RESOLVED and choose a resolution (Fixed, Duplicate, Wontfix, Worksforme).

Verify : Act as a QA tester and move it to VERIFIED or REOPENED.

Goal : Observe how the "Activity" log records every change in status and who made it.

```
import os

import requests
from dotenv import load_dotenv
from rich import print

load_dotenv()

# ----- CONFIG -----
BUGZILLA_URL = "http://localhost:8080/rest"
API_KEY = os.environ.get("API")

PRODUCT = "TestProduct"
COMPONENT = "TestComponent"
VERSION = "unspecified"
# -----

headers = {"Content-Type": "application/json", "X-BUGZILLA-API-KEY": API_KEY}

def login():
    url = f"{BUGZILLA_URL}/login?login=admin@bugzilla.test&password=password01!"
    r = requests.get(url)
    return r.json()["token"]

def get_bug(id, token):
    url = f"{BUGZILLA_URL}/bug/{id}"
    r = requests.get(url, params={"token": token})
    return r.json()

def create_bug(summary, token):
    url = f"{BUGZILLA_URL}/bug"

    data = {
        "product": PRODUCT,
        "component": COMPONENT,
        "summary": summary,
        "version": VERSION,
        "description": "Bug lifecycle test",
        "severity": "major",
        "status": "UNCONFIRMED",
        "priority": "Highest",
        "op_sys": "All",
        "platform": "All",
    }
```

```
r = requests.post(url, params={"token": token}, json=data, headers=headers)

if r.status_code != 200:
    print("Create failed:", r.text)
    return None

bug_id = r.json()["id"]
print("Created bug:", bug_id)
return bug_id

def update_bug(bug_id, data, label, token):
    url = f"{BUGZILLA_URL}/bug/{bug_id}"

    r = requests.put(url, params={"token": token}, json=data, headers=headers)

    if r.status_code != 200:
        print(f"{label} failed:", r.text)
        return False

    print(f"{label} OK")
    return True

def get_history(bug_id, token):
    url = f"{BUGZILLA_URL}/bug/{bug_id}/history"
    r = requests.get(url, params={"token": token}, headers=headers)

    if r.status_code != 200:
        print("History fetch failed:", r.text)
        return

    history = r.json()["bugs"][0]["history"]
    # print(history)

    print("\nActivity Log:")
    print("-----")
    for entry in history:
        for change in entry["changes"]:
            print(
                f"{change['field_name']} -> {change['added']} (was {change['removed']})"
            )

# %%

token = login()

# %%

bug_id = create_bug("Lifecycle test bug", token)

if not bug_id:
    exit()
```

```
# %%

# ASSIGN
update_bug(bug_id, {"status": "CONFIRMED"}, "Assign", token)
info = get_bug(bug_id, token)
print("Changed bug status to", info["bugs"][0]["status"])

# %%

# RESOLVE
update_bug(bug_id, {"assigned_to": "admin@bugzilla.test"}, "Assign to user", token)
update_bug(bug_id, {"status": "RESOLVED", "resolution": "FIXED"}, "Resolve", token)
info = get_bug(bug_id, token)
print("Changed resolution to", info["bugs"][0]["resolution"])
print("Changed status to", info["bugs"][0]["status"])
print("Changed assigned_to to", info["bugs"][0]["assigned_to"])

# %%

# VERIFY
update_bug(bug_id, {"status": "VERIFIED"}, "Verify", token)
info = get_bug(bug_id, token)
print("Changed status to", info["bugs"][0]["status"])

# %%

# Fetch activity log
get_history(bug_id, token)
```

```
> uv run bugzilla.py
Created bug: 5
Assign OK
Changed bug status to CONFIRMED
Assign to user OK
Resolve OK
Changed resolution to FIXED
Changed status to RESOLVED
Changed assigned_to to admin@bugzilla.test
Verify OK
Changed status to VERIFIED

Activity Log:
-----
is_confirmed -> 1 (was 0)
status -> RESOLVED CONFIRMED (was CONFIRMED UNCONFIRMED)
resolution -> FIXED (was )
status -> VERIFIED (was RESOLVED)
```

Lab 7

Q9. Bugzilla Basics - Task: Use a demo Bugzilla instance (e.g., landfill.bugzilla.org).

• Exercise:

1. Create an Account: Register a new user and verify the email.
2. Report a Bug: Pick a "broken" feature on a sample website (like ArtOfTesting).
3. Details: Enter the Product, Component, Severity, and Hardware details. Attach a screenshot of the error.
4. Search: Use the "Advanced Search" to find all bugs with a "New" status assigned to a specific user.

Before reporting a bug, please read the [bug writing guidelines](#), please look at the list of [most frequently reported bugs](#), and please [search](#) for the bug.

[Show Advanced Fields](#) (* = Required Field)

* **Product:** TestProduct **Reporter:** tushyagupta81@gmail.com

* **Component:** TestComponent **Component Description:**
This is a test component in the test product database. This ought to be blown away and replaced with real stuff in a finished installation of Bugzilla.

* **Version:** unspecified **Severity:** major
Hardware: x86 **OS:** Other
We've made a guess at your operating system and platform. Please check them and make any corrections if necessary.

* **Summary:** Website theinternet.com: Login page broken

Possible Duplicates:

Bug ID	Summary	Status	
1313	The manufacturer link in the product details page is broken.	RESOLVED INCOMPLETE	Add Me to the CC List
1349	Changing the currency on a product page freezes the website.	UNCONFIRMED	Add Me to the CC List
1402	Product image is broken and does not display on product page.	RESOLVED FIXED	Add Me to the CC List
1632	The main menu is not displayed on the mobile version of the website.	UNCONFIRMED	Add Me to the CC List
1677	Website ABC: Broken Bug.	UNCONFIRMED	Add Me to the CC List
1679	Login button not responding on sample website	UNCONFIRMED	Add Me to the CC List
1682	Broken Bug	UNCONFIRMED	Add Me to the CC List

Description: **Comment** **Preview**

Attachment: Don't add an attachment
Add an attachment
File: Enter the path to the file on your computer (or [paste text as attachment](#)).
Choose file: screenshot.png
(File size limit: 4096 KB)
* **Description:** Describe the attachment briefly.
Broken login webpage

[Submit Bug](#)

[Home](#) | [New](#) | [Browse](#) | [Search](#) | [Search](#) [?] | [Reports](#) | [My Requests](#) | [Preferences](#) | [Help](#) | [Log out](#) tushyagupta81@gmail.com
[My Bugs](#)

ID	Product	Comp	Assignee	Status	Resolution	Summary	Changed
806	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	BSIT42 Taboy	2025-10-08
810	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	BSIT42 Eulin	2025-10-08
1623	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	ATM PIN trial is blocked	2026-01-22
814	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	BSIT	2025-10-08
975	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	[Exploratory] User cannot add new product in the catalog when logged in as admin	2026-02-02
815	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	BSIT4A1	2025-10-10
1179	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	Приложение полностью не запускается, появляясь черный экран после слеша-скрина	2025-11-14
821	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	bat	2025-10-10
1241	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	Приложение аварийно завершает работу при построении маршрута	2025-11-14
1286	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	Ошибка при отправке сообщения: сервер недоступен	2025-11-14
793	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	BSIT4A1	2025-10-08
794	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	BSIT4A1	2025-10-08
337	TestProd	TestComp	justdave@bugzilla.org	UNCO	---	Bug in Test Component	2025-07-21

Q10. Selenium Fundamentals - Task: Write a script to automate a login flow on a practice site.

• Exercise:

1. Locators: Use By.id, By.name, or XPath to find the username, password, and login button.
2. Actions: Use .sendKeys() to enter credentials and .click() to submit.
3. Validation: Use an Assertion to check if the "Welcome" message appears. If it doesn't, capture a screenshot.

```
from selenium.webdriver import Chrome
from selenium.webdriver.common.by import By

driver = Chrome()

# %%

driver.get("https://the-internet.herokuapp.com/login")

# %%

input_el = driver.find_element(By.ID, "username")
password_el = driver.find_element(By.ID, "password")

input_el.send_keys("tomsmith")
password_el.send_keys("SuperSecretPassword!")

# %%

submit_btn = driver.find_element(By.XPATH, "/html/body/div[2]/div/div/form/button")
submit_btn.click()

# %%

succ_text = driver.find_element(By.XPATH, '//*[@id="content"]/div/h2').text

if succ_text != "Secure Area":
    sc = driver.get_screenshot_as_png()
    with open("screenshot.png", "wb") as f:
        f.write(sc)
```

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username
tomsmith

Password

 Login

 You logged into a secure area!

Secure Area

Welcome to the Secure Area. When you are done click logout below.

Logout

Q11. Integration – The "Auto-Bug" Reporter - Task: Create a Selenium script that automatically logs into Bugzilla and files a bug report when a test fails.

• Exercise:

1. The Trigger: Run a Selenium test on a "faulty" web page.
2. The Catch: Use a try-catch block. If the test fails (e.g., an element is not found), the catch block should trigger a new Selenium routine.
3. The Automation: The "catch" routine must:
 - Navigate to the Bugzilla "Enter Bug" page.
 - Automatically fill in the "Summary" with the error message.
 - Paste the URL of the failing page into the "Description."
 - Submit the bug.

```
from selenium.webdriver import Chrome
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as ec

def report_to_bugzilla(driver, error_msg, url):
    driver.get("http://localhost:8080/enter_bug.cgi")
    email_el = driver.find_element(By.ID, "Bugzilla_login")
    password_el = driver.find_element(By.ID, "Bugzilla_password")
    login_btn = driver.find_element(By.ID, "log_in")

    email_el.send_keys("admin@bugzilla.test")
    password_el.send_keys("password01!")
    login_btn.click()
    WebDriverWait(driver, 5).until(ec.presence_of_element_located((By.ID, "short_desc")))

    summary_el = driver.find_element(By.ID, "short_desc")
    desc_el = driver.find_element(By.ID, "comment")

    summary_el.send_keys(error_msg)
    desc_el.send_keys(url)

    submit_btn = driver.find_element(By.ID, "commit")
    submit_btn.click()

###

driver = Chrome()

# %%

url = "https://the-internet.herokuapp.com/login"
driver.get(url)

# %%

input_el = driver.find_element(By.ID, "username")
password_el = driver.find_element(By.ID, "password")

input_el.send_keys("omsmith")
password_el.send_keys("SuperSecretPassword!")

# %%

submit_btn = driver.find_element(By.XPATH, "/html/body/div[2]/div/div/form/button")
submit_btn.click()

# %%
```

```
succ_text = driver.find_element(By.XPATH, '//*[@id="content"]/div/h2').text

if succ_text != "Secure Area":
    report_to_bugzilla(driver, "Login failed", url)
```

Enter Bug: TestProduct x +

localhost:8080/enter_bug.cgi

Chrome is being controlled by automated test software

Home | New | Browse | Search | Search [?] | Reports | Preferences | Administration | Help | Log out admin@bugzilla.t

Before reporting a bug, please read the [bug writing guidelines](#), please look at the list of [most frequently reported bugs](#), and please [Show Advanced Fields](#) (* = Required Field)

* **Product:** TestProduct **Reporter:** admin@bugzilla.test

* **Component:** TestComponent **Component Description:** This is a test component in the test product database. This ought to be blown away and replaced.

* **Version:** unspecified **Severity:** enhancement

Hardware: PC **OS:** Mac OS

We've made a guess at your operating system and platform. Please check them and make any corrections if necessary.

* **Summary:** Login failed

Possible Duplicates:

Bug ID	Summary	Status
No possible duplicates found.		

Description: **Comment** **Preview**

https://the-internet.herokuapp.com/login

Attachment: Add an attachment

Submit Bug

Bug 7 has been successfully created
Email sent to: no one

Bug 7 - Login failed (edit)

Status: CONFIRMED (edit)

Alias: None (edit)

Product: TestProduct

Component: TestComponent (show other bugs)

Version: unspecified

Hardware: PC Mac OS

Importance: --- enhancement

Assignee: Test Admin (edit)

URL:

Personal Tags:

Depends on:

Blocks:

Q12. Advanced – Data-Driven Testing & APIs - Task: Perform data-driven testing using an Excel sheet and Bugzilla's REST API.

- Exercise:

1. Data-Driven: Create an Excel file with multiple "invalid" login credentials. Use Selenium to loop through them and verify that the correct "Login Failed" error appears for each.
2. API Challenge: Instead of using Selenium to type into Bugzilla, have students use Python Requests or Java REST Assured to submit the bug report via the Bugzilla REST API v1.

```
import os
import time

import pandas as pd
import requests
from dotenv import load_dotenv
from rich import print
from selenium.webdriver import Chrome
from selenium.webdriver.common.by import By

load_dotenv()

# ----- CONFIG -----
BUGZILLA_URL = "http://localhost:8080/rest"
API_KEY = os.environ.get("API")

PRODUCT = "TestProduct"
COMPONENT = "TestComponent"
VERSION = "unspecified"
# -----

headers = {"Content-Type": "application/json", "X-BUGZILLA-API-KEY": API_KEY}

def login_bug():
    url = f"{BUGZILLA_URL}/login?login=admin@bugzilla.test&password=password01!"
    r = requests.get(url)
    return r.json()["token"]

def get_bug(id, token):
    url = f"{BUGZILLA_URL}/bug/{id}"
    r = requests.get(url, params={"token": token})
    return r.json()

def create_bug(summary, token):
    url = f"{BUGZILLA_URL}/bug"

    data = {
        "product": PRODUCT,
        "component": COMPONENT,
        "summary": summary,
        "version": VERSION,
        # "description": "Bug lifecycle test",
        "severity": "major",
        "status": "UNCONFIRMED",
        "priority": "Highest",
        "op_sys": "All",
        "platform": "All",
```

```
}

r = requests.post(url, params={"token": token}, json=data, headers=headers)

if r.status_code != 200:
    print("Create failed:", r.text)
    return None

bug_id = r.json()["id"]
print("Created bug:", bug_id)
return bug_id

token = login_bug()

driver = Chrome()

# %%

df = pd.read_csv("./lab6/creds.csv")

# %%

def login(username, password):
    driver.get("https://the-internet.herokuapp.com/login")

    input_el = driver.find_element(By.ID, "username")
    password_ed = driver.find_element(By.ID, "password")

    input_el.send_keys(username)
    password_ed.send_keys(password)

    submit_btn = driver.find_element(By.XPATH, "/html/body/div[2]/div/div/form/button")

    submit_btn.click()
    time.sleep(1)

    failed_text = driver.find_element(By.ID, "flash").text

    if "invalid" not in failed_text:
        print(f"Logged in for {username}, {password}")
        create_bug(f"Logged in for {username}, {password}", token)

# %%

for r in df.iloc[:, :].values:
    login(r[0], r[1])
```